1. Semantic Concepts and Attribute Grammar

Two types: internal nodes (composite concepts) or leaves (atomic concepts). Terminal nodes are irreducible, so they can become leaves. The exception to this is structural terminals like parentheses or semicolons, which get discarded. Non-terminals that represent a meaningful semantic operation become composite concepts. Non-terminals that are just structural pass-throughs get collapsed and discarded.

1.1. Atomic Semantic Concepts List

Concept	Source Token(s)	AST Leaf Description (Value)
`id`	`id token`	Leaf: stores lexeme (variable, class, or function name) and source location
`intNum`	intnum token	Leaf: stores integer literal value and source location
`floatNum`	floatnum token	Leaf: stores float literal value and source location
`type`	`integer | float | id`	Leaf: stores type keyword or class-type name
`visibility`	`public | private`	Leaf: stores visibility modifier keyword
`relOp`	eq | neq | lt | gt | leq | geq	Leaf: stores relational operator
`addOp`	`+ | - | or`	Leaf: stores additive operator
`multOp`	`* | / | and`	Leaf: stores multiplicative operator
`sign`	`+ | -`	Leaf: stores unary sign
`void`	`void keyword`	Leaf: marks void return type
`epsilon`	`(no token)`	Leaf: epsilon marker for list boundaries, or absent optional element

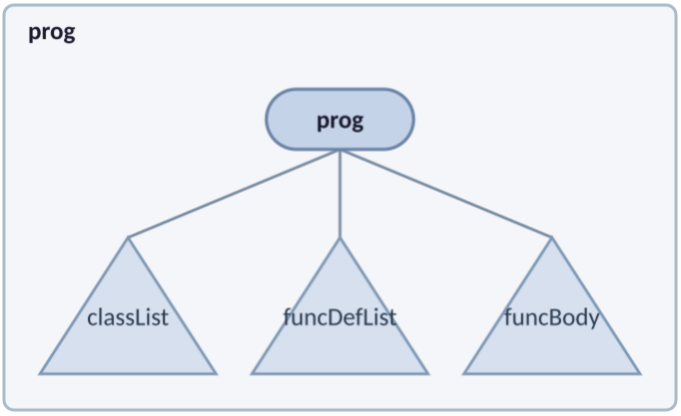
1.2. Composite Semantic Concept List

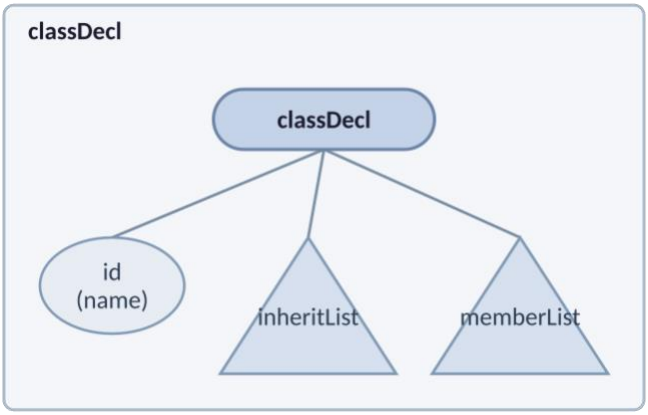
Concept	Children (subordinate concepts)	# Children	Description
prog	classList, funcDefList, mainFuncBody	3	Top-level program node
classList	classDecl*	0..n	List of class declarations
funcDefList	funcDef*	0..n	List of function definitions
classDecl	id(name), inheritList, memberList	3	Single class declaration
inheritList	id*	0..n	List of inherited class names
memberList	memberDecl*	0..n	List of class members

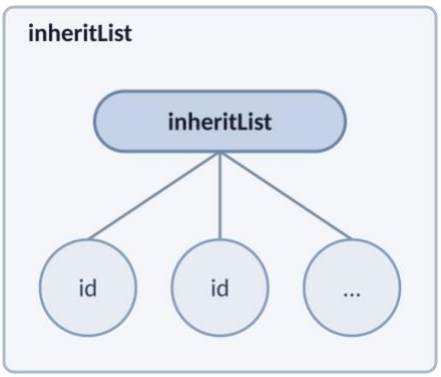
memberDecl	visibility, (funcDecl | varDecl)	2	Class member with its visibility
funcDecl	id(name), fParamsList, returnType	3	Function declaration (in class)
funcDef	funcHead, funcBody	2	Function definition
funcHead	id(scope)|epsilon, id(name), fParamsList, returnType	4	Function header; epsilon scope for free functions
funcBody	varDeclList, statList	2	Function body
varDeclList	varDecl*	0..n	Local variable declarations
varDecl	type, id(name), dimList	3	Variable declaration
dimList	(intNum|epsilon)*	0..n	Array dimension sizes (prof: "array size")
fParamsList	fParam*	0..n	Formal parameter list
fParam	type, id(name), dimList	3	Single formal parameter
aParamsList	expr*	0..n	Actual parameter (argument) list
statList	statement*	0..n	Statement list
statBlock	statement*	0..n	Statement block (do..end, single, or empty)
assignStat	variable, expr	2	Assignment statement
ifStat	relExpr, statBlock(then), statBlock(else)	3	If-then-else statement
whileStat	relExpr, statBlock	2	While loop
readStat	variable	1	Read statement
writeStat	expr	1	Write statement
returnStat	expr	1	Return statement
relExpr	arithExpr, relOp, arithExpr	3	Relational expression
addOp (node)	leftExpr, rightExpr	2	Additive operation (op stored in node)
multOp (node)	leftExpr, rightExpr	2	Multiplicative operation (op stored in node)
not	factor	1	Logical negation
signNode	sign, factor	1	Unary sign (sign value stored in node)
var	id, indiceList	2	Variable element (one segment of dot chain)
dot	left, right	2	Dot access: left.right (left-associative)
indiceList	arithExpr*	0..n	Array indices (prof: "ndc list" / "indices list")
functionCall	id(name), aParamsList	2	Function call

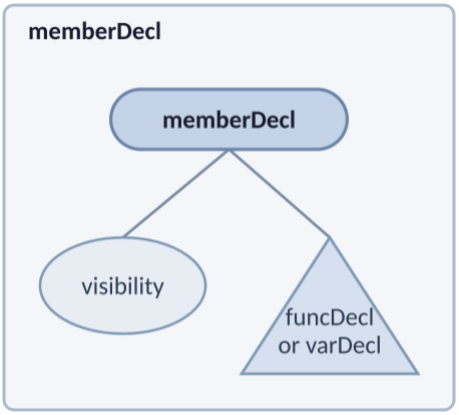
1.3. AST Subtree Relationships

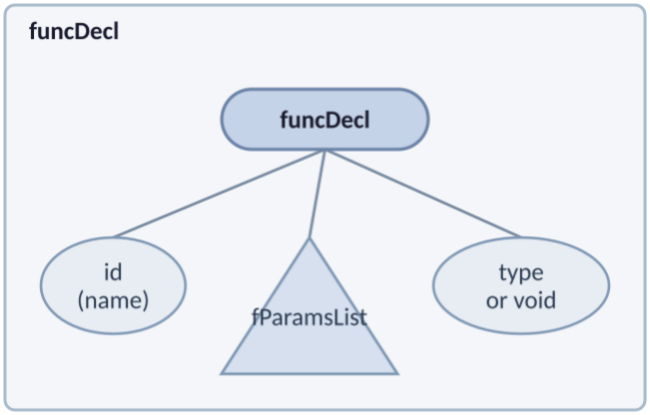
- Rounded Rectangles: Parent Node
- Triangles: Subtrees that are children to the parent node.
- Circles/Ovals: Leaf nodes.

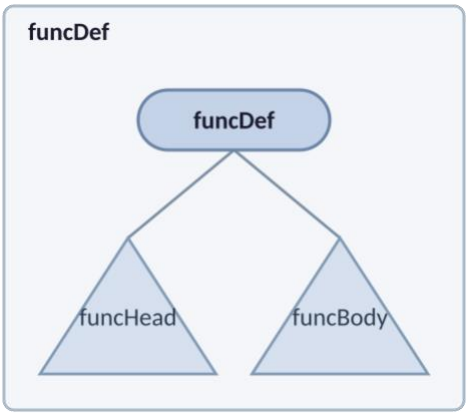


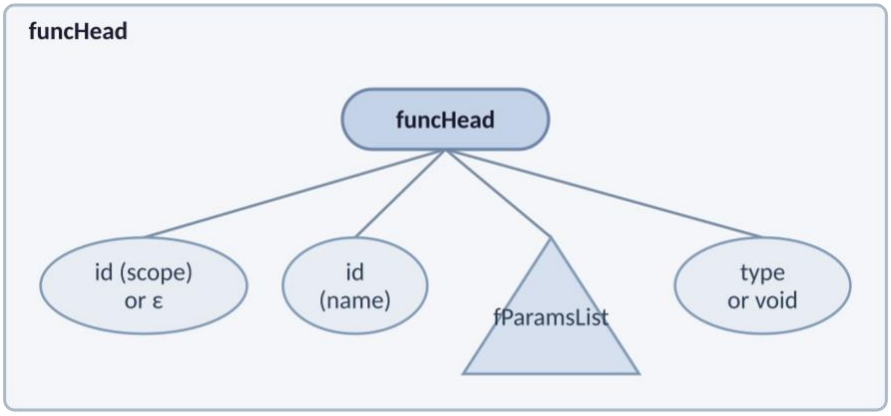


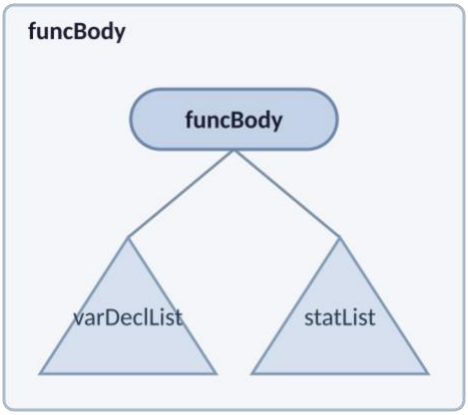


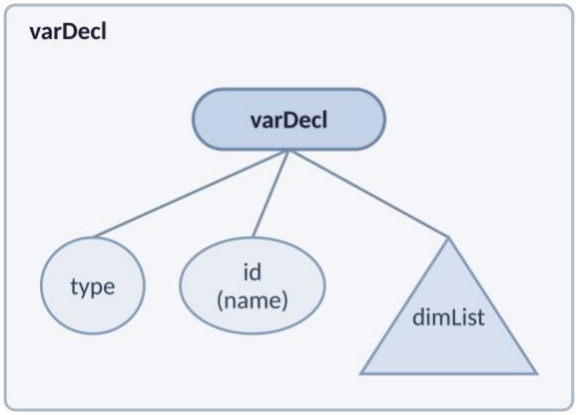


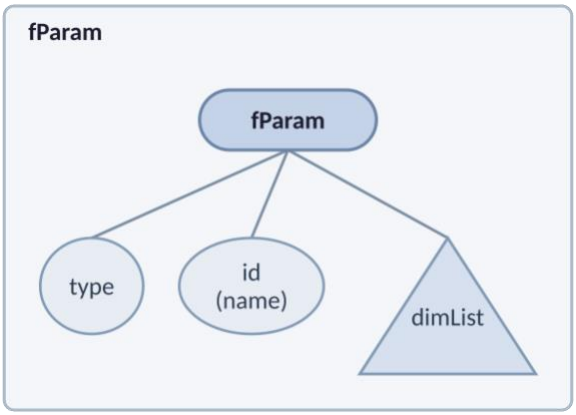


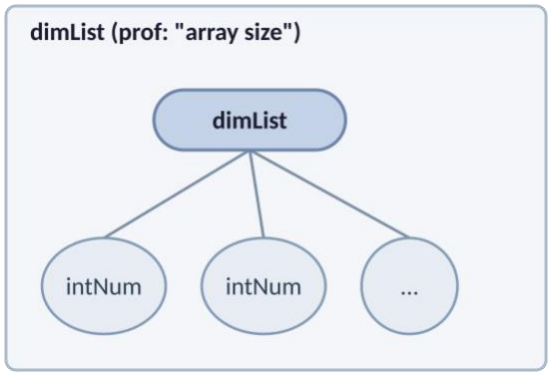

funcDef
funcDef
funcHead
funcBody

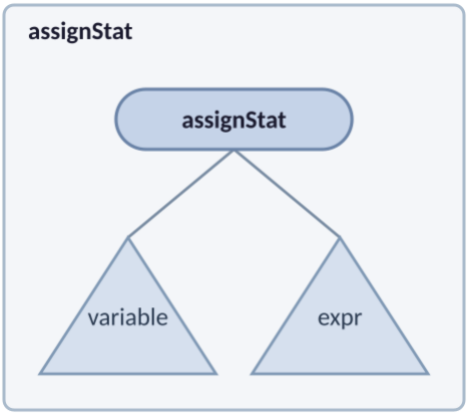

funcHead
funcHead
id (scope)
or ε
id
(name)
fParamsList
type
or void

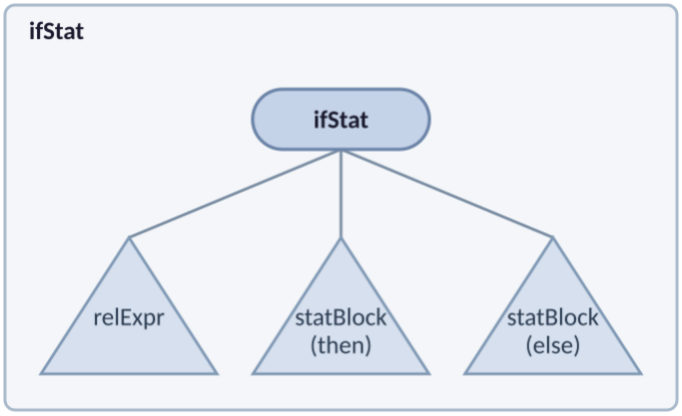

funcBody
funcBody
varDeclList
statList

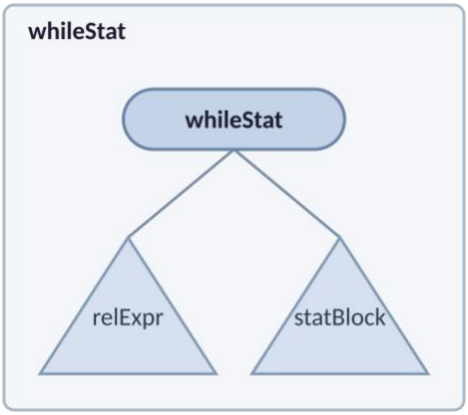

varDecl
varDecl
type
id
(name)
dimList

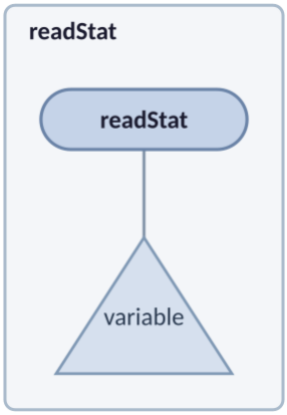

fParam
fParam
type
id
(name)
dimList

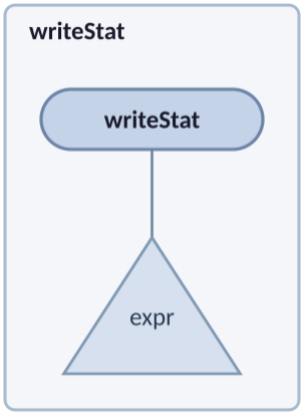

dimList (prof: "array size")
dimList
intNum
intNum
...

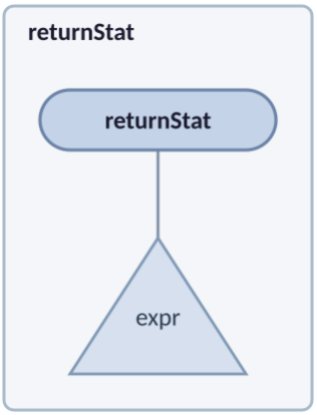

assignStat
assignStat
variable
expr

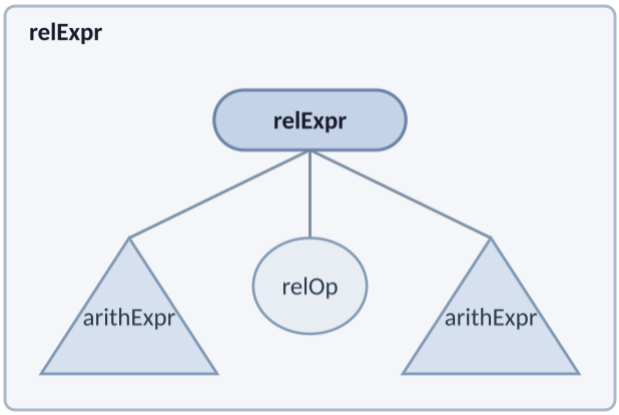

ifStat
ifStat
relExpr
statBlock
(then)
statBlock
(else)

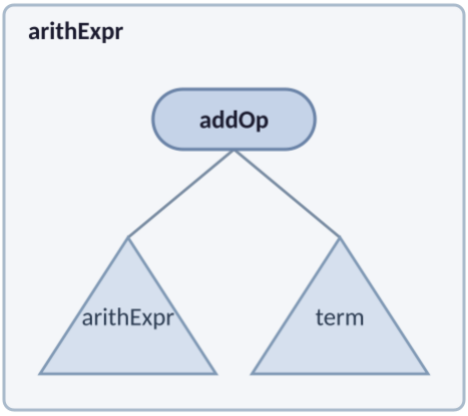

whileStat
whileStat
relExpr
statBlock

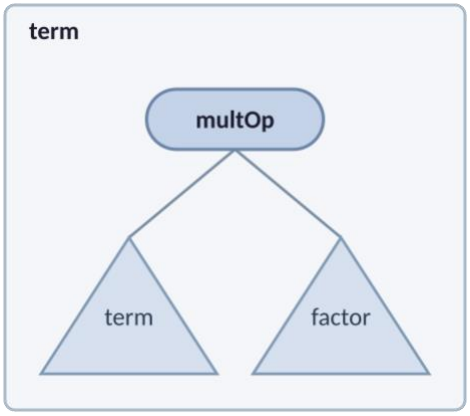

readStat
readStat
variable

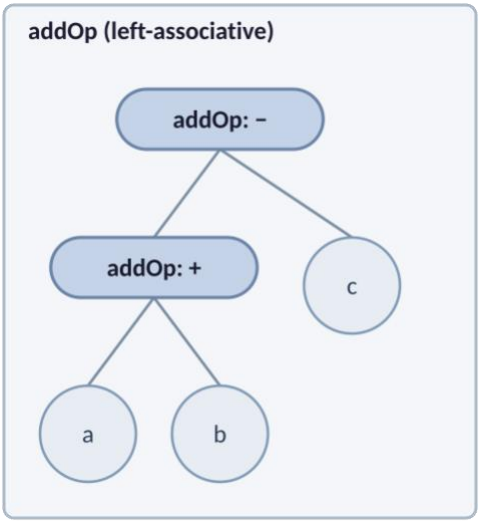

writeStat
writeStat
expr

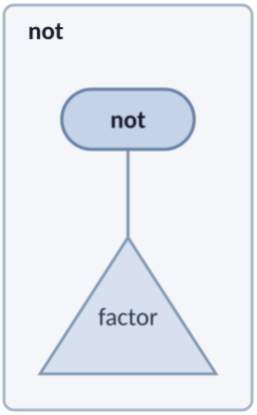

returnStat
returnStat
expr

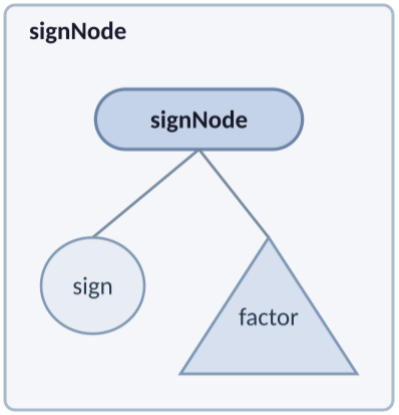

relExpr
relExpr
arithExpr
relOp
arithExpr

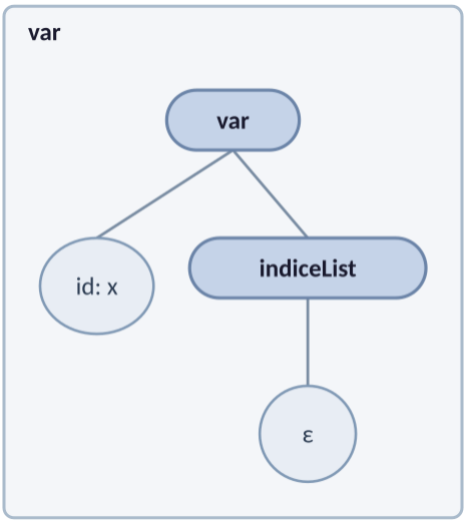

arithExpr
addOp
arithExpr
term

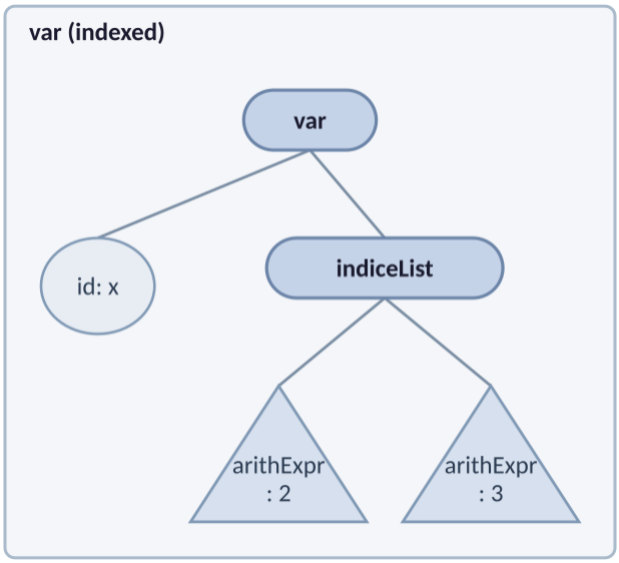

term
multOp
term
factor

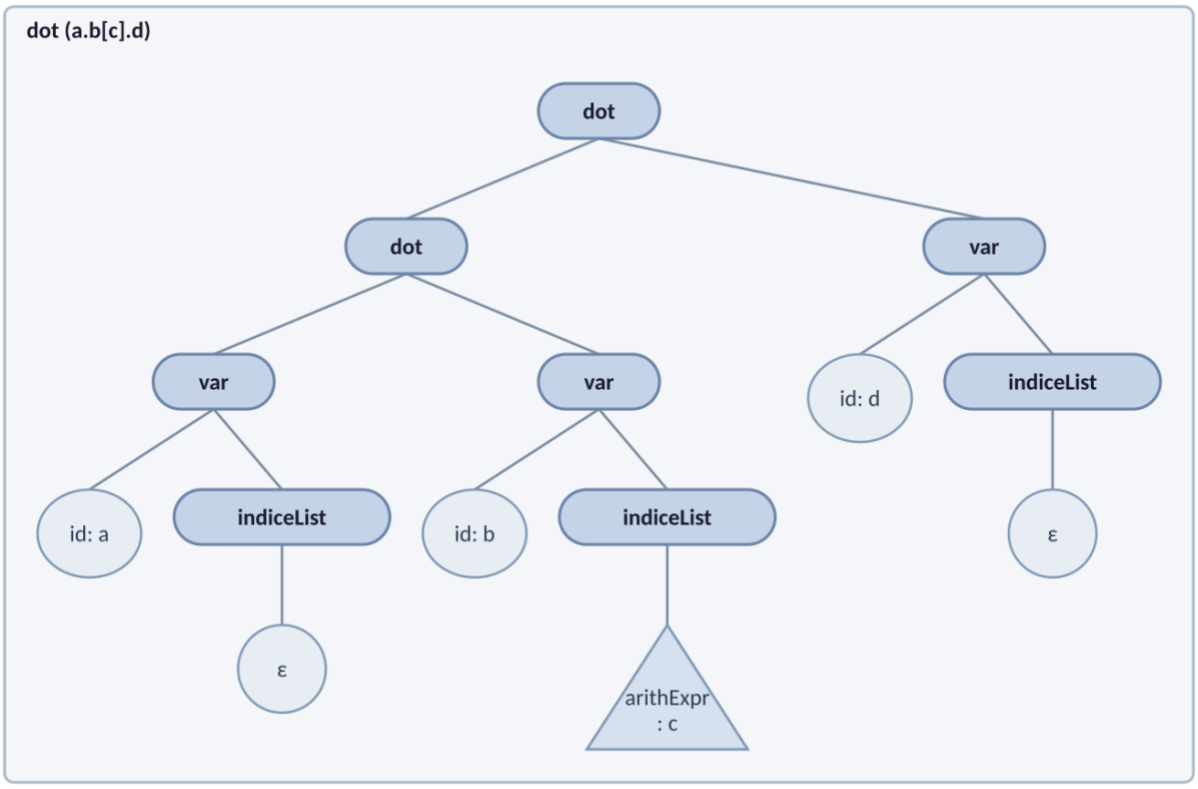

addOp (left-associative)
addOp: –
addOp: +
c
a
b

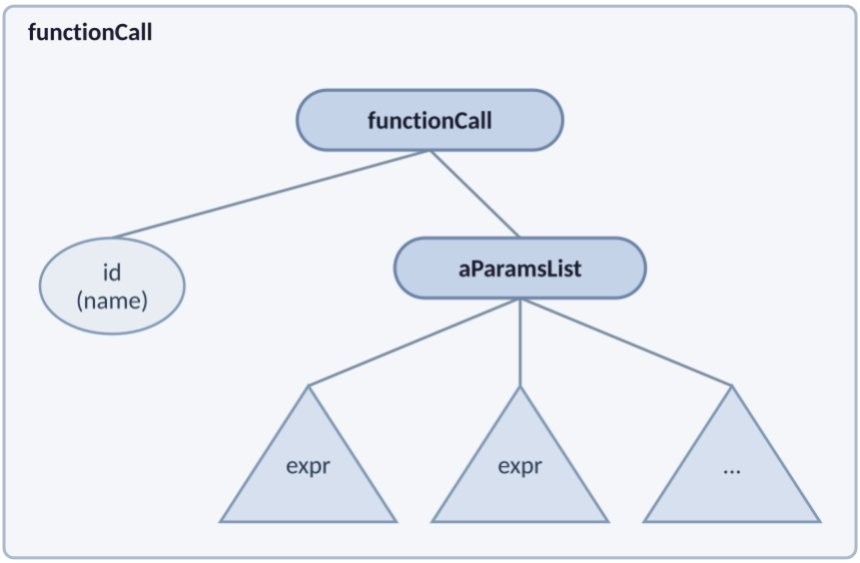

not
not
factor

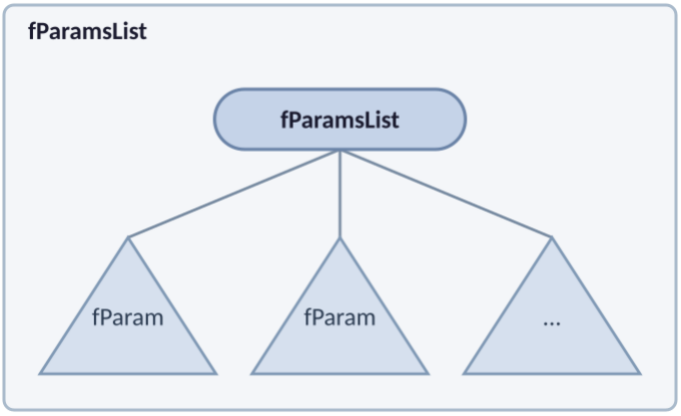

signNode
signNode
sign
factor


var
var
id: x
indiceList
ε


var (indexed)
var
id: x
indiceList
arithExpr
: 2
arithExpr
: 3


dot (a.b[c].d)
dot
dot
var
var
var
id: a
indiceList
ε
id: b
indiceList
arithExpr
: c
id: d
indiceList
ε


functionCall
functionCall
id
(name)
aParamsList
expr
expr
...


fParamsList
fParamsList
fParam
fParam
...

1.4. Attribute Grammar

Following the same notation in slide 8.5, below is the original grammar (not LL1) with semantic actions inserted.

- **Green**: Leaf Node + Push Epsilon
- **Red**: Subtree

For each semantic concept, we define a semantic action that manipulates the **semantic stack**.

Leaf Node Creators (atomic concepts)

Label	Action	Category	Effect
V1	`@MAKE_ID`	Leaf	Create id leaf from last matched token (lexeme + location). Push.
A2, L1	`@MAKE_INTNUM`	Leaf	Create `intNum` leaf from last matched token. Push.
L2	`@MAKE_FLOATNUM`	Leaf	Create `floatNum` leaf from last matched token. Push.
T	`@MAKE_TYPE`	Leaf	Create type leaf (integer/float/id) from last matched token. Push.
V2	`@MAKE_VISIBILITY`	Leaf	Create visibility leaf (public/private) from last matched token. Push.
O1	`@MAKE_ADDOP`	Leaf	Create `addOp` leaf (+/-/or) from last matched token. Push.
M1	`@MAKE_MULTOP`	Leaf	Create `multOp` leaf (*/div/and) from last matched token. Push.
R1	`@MAKE_RELOP`	Leaf	Create `relOp` leaf (eq/neq/lt/gt/leq/geq). Push.
L3	`@MAKE_SIGN`	Leaf	Create sign leaf (+/-) from last matched token. Push.
V4	`@MAKE_VOID`	Leaf	Create void leaf from matched void keyword. Push.
	`@MAKE_NODE_EPSILON`	Leaf	Create epsilon leaf (unsized array []). Push.

Epsilon Marker (push epsilon)

Label	Action	Category	Effect
E1–E10	`@PUSH_EPSILON`	Epsilon marker	`Push epsilon marker onto semantic stack. Boundary for pop-until-epsilon.`

Subtree Creators , Fixed Number of Children

Label	Action	Category	Effect
C3	@MAKE_PROG	Pop 3	Pop 3: classList, funcDefList, mainFuncBody. makeFamily. Push.
C7	@MAKE_CLASSDECL	Pop 3	Pop 3: className, inheritList, memberList. makeFamily. Push.
C5	@MAKE_MEMBERDECL	Pop 2	Pop 2: visibility, declaration. makeFamily. Push.
F1	@MAKE_FUNCDECL	Pop 3	Pop 3: funcName, fParamsList, returnType. makeFamily. Push.
D1	@MAKE_FUNCDEF	Pop 2	Pop 2: funcHead, funcBody. makeFamily. Push.
H1	@MAKE_FUNCHEAD	Pop 4	Pop 4: scope, funcName, fParamsList, returnType. makeFamily. Push.
B3	@MAKE_FUNCBODY	Pop 2	Pop 2: varDeclList, statList. makeFamily. Push.
V3	@MAKE_VARDECL	Pop 3	Pop 3: type, varName, dimList. makeFamily. Push.
P1	@MAKE_FPARAM	Pop 3	Pop 3: type, paramName, dimList. makeFamily. Push.
S6	@MAKE_ASSIGNSTAT	Pop 2	Pop 2: variable, expr. makeFamily. Push.
S1	@MAKE_IFSTAT	Pop 3	Pop 3: condition, thenBlock, elseBlock. makeFamily. Push.
S2	@MAKE_WHILESTAT	Pop 2	Pop 2: condition, loopBody. makeFamily. Push.
S3	@MAKE_READSTAT	Pop 1	Pop 1: variable. makeFamily. Push.
S4	@MAKE_WRITESTAT	Pop 1	Pop 1: expr. makeFamily. Push.
S5	@MAKE_RETURNSTAT	Pop 1	Pop 1: expr. makeFamily. Push.
R2	@MAKE_RELEXPR	Pop 3	Pop 3: left, `relOp`, right. makeFamily. Push.
O2	@MAKE_ADDNODE	Pop 3	Pop 3: left, `addOp`, right. makeFamily with op in node. Push as new left operand.
M2	@MAKE_MULTNODE	Pop 3	Pop 3: left, `multOp`, right. makeFamily with op in node. Push as new left operand.
N1	@MAKE_NOT	Pop 1	Pop 1: factor. makeFamily. Push.
N2	@MAKE_SIGNFACTOR	Pop 2	Pop 2: sign, factor. makeFamily. Push.
W1	@MAKE_VAR	Pop 2	Pop 2: id, indiceList. makeFamily. Push.
W2	@MAKE_DOT	Pop 2	Pop 2: left, right. makeFamily. Push. Builds left-associative dot chain.
Q2	@MAKE_FUNCCALL	Pop 2	Pop 2: funcName, aParamsList. makeFamily. Push.
C1	@MAKE_CLASSLIST	Pop until ε	Pop until epsilon. makeSiblings + adoptChildren → classList. Push.
C2	@MAKE_FUNCDEFLIST	Pop until ε	Pop until epsilon → funcDefList. Push.
C4	@MAKE_INHERITLIST	Pop until ε	Pop until epsilon → inheritList. Push.
C6	@MAKE_MEMBERLIST	Pop until ε	Pop until epsilon → memberList. Push.
B1	@MAKE_VARDECLLIST	Pop until ε	Pop until epsilon → varDeclList. Push.
B2	@MAKE_STATLIST	Pop until ε	Pop until epsilon → statList. Push.

S7	**@MAKE_STATBLOCK**	Pop until ε	Pop until epsilon → statBlock. Push.
A3	**@MAKE_DIMLIST**	Pop until ε	Pop until epsilon → dimList. Push.
I1	**@MAKE_INDICELIST**	Pop until ε	Pop until epsilon → indiceList. Push.
P2	**@MAKE_FPARAMSLIST**	Pop until ε	Pop until epsilon → fParamsList. Push.
Q1	**@MAKE_APARAMSLIST**	Pop until ε	Pop until epsilon → aParamsList. Push.

Subtree Creators , Variable Number of Children (pop until epsilon)

Label	Action	Category	Effect
C1	**@MAKE_CLASSLIST**	Pop until ε	Pop until epsilon. makeSiblings + adoptChildren → classList. Push.
C2	**@MAKE_FUNCDEFLIST**	Pop until ε	Pop until epsilon → funcDefList. Push.
C4	**@MAKE_INHERITLIST**	Pop until ε	Pop until epsilon → inheritList. Push.
C6	**@MAKE_MEMBERLIST**	Pop until ε	Pop until epsilon → memberList. Push.
B1	**@MAKE_VARDECLLIST**	Pop until ε	Pop until epsilon → varDeclList. Push.
B2	**@MAKE_STATLIST**	Pop until ε	Pop until epsilon → statList. Push.
S7	**@MAKE_STATBLOCK**	Pop until ε	Pop until epsilon → statBlock. Push.
A3	**@MAKE_DIMLIST**	Pop until ε	Pop until epsilon → dimList. Push.
I1	**@MAKE_INDICELIST**	Pop until ε	Pop until epsilon → indiceList. Push.
P2	**@MAKE_FPARAMSLIST**	Pop until ε	Pop until epsilon → fParamsList. Push.
Q1	**@MAKE_APARAMSLIST**	Pop until ε	Pop until epsilon → aParamsList. Push.

Special Actions

Label	Action	Category	Effect
	@PUSH_NULLSCOPE	Special	`Pop id, push epsilon, push id back. Inserts null scope so @MAKE_FUNCHEAD always pops 4.`
	@RETYPE_TOP_TO_TYPE	Special	Change top-of-stack from 'id' to 'type'. For MEMBERDECL when id is class-type name.

Program Structure

```
<prog>  ::=  E1 {{<classDecl>}} C1  E2 {{<funcDef>}} C2  'main' <funcBody> C3
   E1:  push(ε)                              // epsilon marker before class list
   C1:  push(createSubtree(classList, popUntilε))
   E2:  push(ε)                              // epsilon marker before funcDef list
   C2:  push(createSubtree(funcDefList, popUntilε))
   C3:  push(createSubtree(prog, pop, pop, pop))
```

Class Declaration

```
<classDecl>  ::=  'class' 'id' V1  [['inherits' E3 'id' V1 {{',' 'id' V1}} C4]]  '{' E4
{{<visibility> V2 <memberDecl> C5}} C6 '}' ';' C7
   V1:  push(createLeaf(id))
   V2:  push(createLeaf(visibility))
   E3:  push(ε)                              // epsilon marker before inherit list
   C4:  push(createSubtree(inheritList, popUntilε))
   E4:  push(ε)                              // epsilon marker before member list
   C5:  push(createSubtree(memberDecl, pop, pop))
   C6:  push(createSubtree(memberList, popUntilε))
   C7:  push(createSubtree(classDecl, pop, pop, pop))
```

Variable Declaration

```
<varDecl>  ::=  <type> 'id' V1  A1 {{<arraySize>}} A3  ';'  V3
<type>  ::=  'integer' T  |  'float' T  |  'id' T
<arraySizeRept>  ::=  '[' 'intLit' A2 ']' <arraySizeRept>  |  EPSILON
   V1:  push(createLeaf(id))
   T:  push(createLeaf(type))
   A1:  push(ε)                              // epsilon marker before dim list
   A2:  push(createLeaf(intNum))
   A3:  push(createSubtree(dimList, popUntilε))
   V3:  push(createSubtree(varDecl, pop, pop, pop))
```

Function Declaration (in class)

```
<funcDecl>  ::=  'id' V1  '(' <fParams> ')'  ':' (<type> T | 'void' V4)  ';'  F1
   V1:  push(createLeaf(id))
   T:  push(createLeaf(type))
   V4:  push(createLeaf(void))
   F1:  push(createSubtree(funcDecl, pop, pop, pop))
```

Function Head

```
<funcHead>  ::=  [['id' V1 'sr']]  'id' V1  '(' <fParams> ')'  ':' (<type> T | 'void' V4)
H1
   H1:  push(createSubtree(funcHead, pop, pop, pop, pop))
```

Function Definition

```
<funcDef>  ::=  <funcHead> <funcBody> ';'  D1
   D1:  push(createSubtree(funcDef, pop, pop))
```

Function Body

```
<funcBody>  ::=  '{'  [['local' E5 {{<varDecl>}} B1]]  'do' E6 {{<statement>}} B2 'end'  B3
   E5:  push(ε)                              // epsilon marker before varDecl list
   B1:  push(createSubtree(varDeclList, popUntilε))
   E6:  push(ε)                              // epsilon marker before statement list
   B2:  push(createSubtree(statList, popUntilε))
   B3:  push(createSubtree(funcBody, pop, pop))
```

Formal Parameters

```
<fParams>  ::=  E7  <type> T 'id' V1  A1 {{<arraySize>}} A3  P1  {{<fParamsTail>}}  P2
<fParams>  ::=  EPSILON  →  E7 P2
<fParamsTail>  ::=  ',' <type> T 'id' V1  A1 {{<arraySize>}} A3  P1
   E7:  push(ε)                              // epsilon marker before fParams list
   P1:  push(createSubtree(fParam, pop, pop, pop))
   P2:  push(createSubtree(fParamsList, popUntilε))
```

Statements

```
<statement>  ::=  <assignStat> ';'
               |  'if' '(' <relExpr> ')' 'then' <statBlock> 'else' <statBlock> ';'  S1
               |  'while' '(' <relExpr> ')' <statBlock> ';'  S2
               |  'read' '(' <variable> ')' ';'  S3
               |  'write' '(' <expr> ')' ';'  S4
               |  'return' '(' <expr> ')' ';'  S5
               |  <functionCall> ';'
   S1:  push(createSubtree(ifStat, pop, pop, pop))
   S2:  push(createSubtree(whileStat, pop, pop))
   S3:  push(createSubtree(readStat, pop))
   S4:  push(createSubtree(writeStat, pop))
   S5:  push(createSubtree(returnStat, pop))
```

Assignment

```
<assignStat>  ::=  <variable> '=' <expr>  S6
   S6:  push(createSubtree(assignStat, pop, pop))
```

Statement Block

```
<statBlock>  ::=  'do' E8 {{<statement>}} S7 'end'  |  E8 <statement> S7  |  EPSILON → E8 S7
   E8:  push(ε)
   S7:  push(createSubtree(statBlock, popUntilε))
```

Expressions

```
<expr>  ::=  <arithExpr>  |  <relExpr>
<relExpr>  ::=  <arithExpr> <relOp> R1 <arithExpr>  R2
<arithExpr>  ::=  <arithExpr> <addOp> O1 <term>  O2  |  <term>
<term>  ::=  <term> <multOp> M1 <factor>  M2  |  <factor>
   R1:  push(createLeaf(relOp))
   R2:  push(createSubtree(relExpr, pop, pop, pop))
   O1:  push(createLeaf(addOp))
   O2:  push(createSubtree(addOp, pop, pop, pop))
   M1:  push(createLeaf(multOp))
   M2:  push(createSubtree(multOp, pop, pop, pop))
```

Factor

```
<factor>  ::=  <variable>  |  <functionCall>  |  'intNum' L1  |  'floatNum' L2
            |  '(' <arithExpr> ')'  |  'not' <factor> N1  |  <sign> L3 <factor> N2
   L1:  push(createLeaf(intNum))
   L2:  push(createLeaf(floatNum))
   L3:  push(createLeaf(sign))
   N1:  push(createSubtree(not, pop))
   N2:  push(createSubtree(signNode, pop, pop))
```

Variable (with dot chain)

```
<variable>  ::=  {{<idnest>}} 'id' V1  E9 {{<indice>}} I1  W1
```

```
<idnest>  ::=  'id' V1  E9 {{<indice>}} I1  W1  '.'          // after each segment beyond first: W2
   V1:  push(createLeaf(id))
   E9:  push(ε)
   I1:  push(createSubtree(indiceList, popUntilε))
   W1:  push(createSubtree(var, pop, pop))
   W2:  push(createSubtree(dot, pop, pop))
```

Function Call

```
<functionCall>  ::=  {{<idnest>}} 'id' V1  '(' E10 <aParams> Q1 ')'  Q2
   E10:  push(ε)
   Q1:  push(createSubtree(aParamsList, popUntilε))
   Q2:  push(createSubtree(functionCall, pop, pop))
```

Remaining Rules

```
<indice>  ::=  '[' <arithExpr> ']'                // arithExpr subtree collected by indiceList
<aParams>  ::=  <expr> {{',' <expr>}}        // each expr collected by aParamsList
<visibility>  ::=  'public' V2  |  'private' V2
   V2:  push(createLeaf(visibility))
```

2. Design

2.2 Component Roles

The following components were created or modified for Assignment 3:

Component	File	Role
ParsingTable	ParsingTable.java	Grammar rules augmented with `@SEMANTIC_ACTION` markers at positions identified by the attribute grammar.
Parser	Parser.java	Extended with a **semantic stack** (Deque<ASTNode>) and an action dispatcher (executeSemanticAction()) that builds the AST during parsing.
ASTNode	ASTNode.java	Universal AST node class. Distinguishes leaf vs. composite nodes. Provides factory methods (makeNode, makeFamily).
Visitor	Visitor.java	Interface for AST traversal (Visitor pattern).
ASTPrinter	ASTPrinter.java	Visitor that outputs indented text representation to .outast.
ASTDotPrinter	ASTDotPrinter.java	Visitor that outputs Graphviz DOT graph to .outast.dot.
ASTDriver	ASTDriver.java	Orchestrates the full pipeline: accepts .src files, runs lexer + parser, invokes visitors, writes all output files.

2.3 Parser with Semantic Stack

The core addition is a **semantic stack** alongside the existing parsing stack. The main parse loop handles three cases:

1. **Semantic action** (top starts with `@`): Pop from parsing stack, execute the action on the semantic stack (create leaf, assemble subtree, or push epsilon).
2. **Terminal**: Match against lookahead. On match, save the token in lastMatchedToken so subsequent leaf actions can read its lexeme and line number.
3. **Non-terminal**: Look up the parsing table, push the RHS (including embedded `@` markers) in reverse order.

This keeps parsing logic and semantic logic cleanly separated, the grammar rules embed `@ACTION` markers at the correct positions, and the parser dispatches them without needing to know their semantics.

2.4 AST Node Design

We use a **single ASTNode class** where the node type is a String field, rather than a class hierarchy with subclasses per node type. This was chosen because:

- The language has 40+ AST node types, a class per type would be excessive.
- All nodes share the same structure: a type, an optional value (for leaves), and an optional children list (for composites).
- Factory methods (makeNode for leaves, makeFamily for composites) enforce the leaf/composite distinction at construction time.

Leaf nodes have value != null and children == null (e.g., Id, IntNum, Type). **Composite nodes** have children != null and value == null (e.g., VarDecl, IfStat, Prog). **Operator nodes** (e.g., AddOp, MultOp) are composites that also store the operator symbol in value.

2.5 Semantic Actions

The 40+ semantic actions fall into four categories:

- **Leaf creators** (`@MAKE_ID`, `@MAKE_INTNUM`, etc.): Push a leaf node from the most recently matched token.
- **Epsilon marker** (`@PUSH_EPSILON`): Push a epsilon marker onto the semantic stack to delimit variable-length lists.
- **List collectors** (`@MAKE_DIMLIST`, `@MAKE_CLASSLIST`, etc.): Pop nodes until the epsilon marker, collect them, and push a single list node.
- **Subtree constructors** (`@MAKE_VARDECL`, `@MAKE_IFSTAT`, etc.): Pop a fixed number of children and push a parent node.

Two special actions handle edge cases: - `@PUSH_NULLSCOPE`: Inserts an epsilon scope node for free functions so that `@MAKE_FUNCHEAD always pops exactly 4 children. - @RETYPE_TOP_TO_TYPE`: Changes the top node's type from Id to Type when a class name is used as a type in member declarations.

2.6 Error Recovery and AST Robustness

When error recovery skips grammar symbols, the embedded semantic actions for those symbols are also skipped, leaving the semantic stack with fewer nodes than expected. To handle this, all `pop()` calls in semantic actions use a safePop() helper that returns a placeholder ErrorNode when the stack is empty. This ensures the parser never crashes on erroneous input, it reports errors, generates a partial AST, and continues processing.

2.7 Visitor Pattern for Output

The **Visitor pattern** decouples AST traversal from AST construction. The Visitor interface has a single visit(ASTNode) method. Each visitor performs a recursive depth-first traversal:

- **ASTPrinter**: Outputs indented text where each level is prefixed with "| ". Leaf nodes display their value after " | ". This format matches the assignment's reference examples.
- **ASTDotPrinter**: Generates a Graphviz DOT directed graph with counter-based node IDs, parent-child edges, and labels showing node type and value.

This design makes it straightforward to add new traversals (e.g., for semantic analysis in future assignments) without modifying the AST classes.

2.8 Test Strategy

Testing is organized in three tiers:

1. **Unit tests** (ASTNodeTest.java, ASTPrinterTest.java): Verify node construction, tree building, and printer output in isolation.
2. **Integration tests** (ASTIntegrationTest.java): Parse .src files through the full pipeline and verify the AST structure (node types, child counts, values).
3. **Source-level test cases** (22 .src files in tests/testcases/): Cover every language construct, minimal programs, classes with inheritance, functions, arrays, dot notation, all operators, and error recovery.

3. Use of tools

No tools have been used in this assignment, as augmented grammar is an adhoc approach.